

Exp :5 Write a program for Implementation of K-Nearest Neighbours (K-NN) in Python

NAME: SAMUEL I

REG NO :192312584

Aim

To implement the K-Nearest Neighbours (K-NN) algorithm in Python and classify test data using an appropriate dataset.

Algorithm

1. Load the dataset.
2. Split the dataset into training and testing sets.
3. Choose the value of K.
4. Train the K-NN classifier using the training data.
5. Predict the class labels for the test data.
6. Evaluate the model using accuracy and confusion matrix.
7. Display the results.

Program :

1.

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
# Load dataset
```

```
X, y = load_iris(return_X_y=True)
```

```
# Split data
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42  
)
```

```
# Feature Scaling
```

```
sc = StandardScaler()
```

```
X_train = sc.fit_transform(X_train)
```

```
X_test = sc.transform(X_test)
```

```
# KNN Model
```

```
model = KNeighborsClassifier(n_neighbors=3)
```

```
model.fit(X_train, y_train)
```

```
# Prediction
```

```
y_pred = model.predict(X_test)
```

```
print("Confusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

OUTPUT:

```
Confusion Matrix:  
[[19  0  0]  
 [ 0 13  0]  
 [ 0  0 13]]  
Accuracy: 1.0
```

2.

PROGRAM:

```
from sklearn.datasets import load_wine  
  
from sklearn.model_selection import train_test_split  
  
from sklearn.preprocessing import StandardScaler  
  
from sklearn.neighbors import KNeighborsClassifier  
  
from sklearn.metrics import confusion_matrix, accuracy_score  
  
X, y = load_wine(return_X_y=True)  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42  
)  
  
sc = StandardScaler()  
  
X_train = sc.fit_transform(X_train)  
  
X_test = sc.transform(X_test)
```

```
model = KNeighborsClassifier(n_neighbors=5)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print(confusion_matrix(y_test, y_pred))
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

OUTPUT:

```
[[19  0  0]
 [ 1 19  1]
 [ 0  0 14]]
Accuracy: 0.9629629629629629
```

3.

PROGRAM:

```
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
X, y = load_breast_cancer(return_X_y=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.3, random_state=42  
)
```

```
sc = StandardScaler()
```

```
X_train = sc.fit_transform(X_train)
```

```
X_test = sc.transform(X_test)
```

```
model = KNeighborsClassifier(n_neighbors=5)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print(confusion_matrix(y_test, y_pred))
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

OUTPUT:

```
[[ 59   4]  
 [   3 105]]  
Accuracy: 0.9590643274853801
```

4.

PROGRAM:

```
from sklearn.datasets import load_digits
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import confusion_matrix, accuracy_score


X, y = load_digits(return_X_y=True)


X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.3, random_state=42

)


sc = StandardScaler()

X_train = sc.fit_transform(X_train)

X_test = sc.transform(X_test)


model = KNeighborsClassifier(n_neighbors=3)

model.fit(X_train, y_train)


y_pred = model.predict(X_test)


print(confusion_matrix(y_test, y_pred))

print("Accuracy:", accuracy_score(y_test, y_pred))
```

OUTPUT:

```
• [[53  0  0  0  0  0  0  0  0  0]
   [ 0 50  0  0  0  0  0  0  0  0]
   [ 0  1 46  0  0  0  0  0  0  0]
   [ 0  0  1 51  0  0  0  0  1  1]
   [ 0  1  0  0 59  0  0  0  0  0]
   [ 0  0  0  0  0 64  1  0  0  1]
   [ 0  0  0  0  0  0 53  0  0  0]
   [ 0  0  0  0  0  0  0 54  0  1]
   [ 0  1  1  1  0  0  0  0 40  0]
   [ 0  0  0  2  1  1  1  0  1 53]]
Accuracy: 0.9685185185185186
```

5.

PROGRAM:

```
from sklearn.datasets import make_classification
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
X, y = make_classification(
```

```
    n_samples=1000,
```

```
    n_features=8,
```

```
    n_informative=5,
```

```
    n_classes=2,
```

```
    random_state=42
```

```
)
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42  
)
```

```
sc = StandardScaler()
```

```
X_train = sc.fit_transform(X_train)
```

```
X_test = sc.transform(X_test)
```

```
model = KNeighborsClassifier(n_neighbors=5)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print(confusion_matrix(y_test, y_pred))
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

OUTPUT:

```
[[133  6]  
 [ 21 140]  
Accuracy: 0.91
```


Result

The K-Nearest Neighbours (K-NN) algorithm was successfully implemented in Python using the Iris dataset. The classifier was trained and tested, and the classification accuracy was obtained successfully